

```
In [9]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [10]: df=pd.read_csv("Iris.csv")
df.tail()
#df['Species'].unique()
```

```
Out[10]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

```
In [3]: df.isnull().sum()
```

```
Out[3]: Id          0
SepalLengthCm      0
SepalWidthCm       0
PetalLengthCm      0
PetalWidthCm       0
Species            0
dtype: int64
```

```
In [4]: df.dtypes
```

```
Out[4]: Id          int64
SepalLengthCm    float64
SepalWidthCm     float64
PetalLengthCm    float64
PetalWidthCm     float64
Species          object
dtype: object
```

```
In [5]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Id              150 non-null   int64
 1   SepalLengthCm  150 non-null   float64
 2   SepalWidthCm   150 non-null   float64
 3   PetalLengthCm  150 non-null   float64
 4   PetalWidthCm   150 non-null   float64
 5   Species        150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB

```

In [6]: `df.describe()`

Out[6]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

In [16]: *# now preparing our model as per Gaussian Naive Bayesian*

```

X = df.iloc[:,1:5] # X is the features in our dataset
y = df.iloc[:, -1] # y is the Labels in our dataset

from sklearn.naive_bayes import GaussianNB

```

In [17]: `from sklearn.model_selection import train_test_split`

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_st

```

In [18]: `from sklearn.naive_bayes import GaussianNB`

```

model = GaussianNB().fit(X_train, y_train)

```

In [19]: `y_pred= model.predict(X_test) #now predicting our model to our test dataset`

```

print(y)
print(y_pred)

```

```

0      Iris-setosa
1      Iris-setosa
2      Iris-setosa
3      Iris-setosa
4      Iris-setosa
...
145    Iris-virginica
146    Iris-virginica
147    Iris-virginica
148    Iris-virginica
149    Iris-virginica
Name: Species, Length: 150, dtype: object
['Iris-versicolor' 'Iris-setosa' 'Iris-virginica' 'Iris-versicolor'
 'Iris-versicolor' 'Iris-setosa' 'Iris-versicolor' 'Iris-virginica'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-virginica'
 'Iris-virginica' 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica'
 'Iris-setosa' 'Iris-virginica' 'Iris-setosa' 'Iris-virginica'
 'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-versicolor'
 'Iris-setosa' 'Iris-setosa' 'Iris-virginica' 'Iris-versicolor'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-virginica'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-setosa' 'Iris-setosa'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica' 'Iris-versicolor'
 'Iris-virginica']

```

```
In [12]: from sklearn.metrics import accuracy_score
```

```

# now calculating that how much accurate our model is with comparing our predicted
accuracy_score = accuracy_score(y_test, y_pred)
print (accuracy_score)

```

```
0.96
```

```
In [20]: X_train
```

Out[20]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
96	5.7	2.9	4.2	1.3
105	7.6	3.0	6.6	2.1
66	5.6	3.0	4.5	1.5
0	5.1	3.5	1.4	0.2
122	7.7	2.8	6.7	2.0
...	...	...	...	...
71	6.1	2.8	4.0	1.3
106	4.9	2.5	4.5	1.7
14	5.8	4.0	1.2	0.2
92	5.8	2.6	4.0	1.2
102	7.1	3.0	5.9	2.1

100 rows × 4 columns

In [21]: `x_test`

Out[21]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
73	6.1	2.8	4.7	1.2
18	5.7	3.8	1.7	0.3
118	7.7	2.6	6.9	2.3
78	6.0	2.9	4.5	1.5
76	6.8	2.8	4.8	1.4
31	5.4	3.4	1.5	0.4
64	5.6	2.9	3.6	1.3
141	6.9	3.1	5.1	2.3
68	6.2	2.2	4.5	1.5
82	5.8	2.7	3.9	1.2
110	6.5	3.2	5.1	2.0
12	4.8	3.0	1.4	0.1
36	5.5	3.5	1.3	0.2
9	4.9	3.1	1.5	0.1
19	5.1	3.8	1.5	0.3
56	6.3	3.3	4.7	1.6
104	6.5	3.0	5.8	2.2
69	5.6	2.5	3.9	1.1
55	5.7	2.8	4.5	1.3
132	6.4	2.8	5.6	2.2
29	4.7	3.2	1.6	0.2
127	6.1	3.0	4.9	1.8
26	5.0	3.4	1.6	0.4
128	6.4	2.8	5.6	2.1
131	7.9	3.8	6.4	2.0
145	6.7	3.0	5.2	2.3
108	6.7	2.5	5.8	1.8
143	6.8	3.2	5.9	2.3
45	4.8	3.0	1.4	0.3
30	4.8	3.1	1.6	0.2

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
22	4.6	3.6	1.0	0.2
15	5.7	4.4	1.5	0.4
65	6.7	3.1	4.4	1.4
11	4.8	3.4	1.6	0.2
42	4.4	3.2	1.3	0.2
146	6.3	2.5	5.0	1.9
51	6.4	3.2	4.5	1.5
27	5.2	3.5	1.5	0.2
4	5.0	3.6	1.4	0.2
32	5.2	4.1	1.5	0.1
142	5.8	2.7	5.1	1.9
85	6.0	3.4	4.5	1.6
86	6.7	3.1	4.7	1.5
16	5.4	3.9	1.3	0.4
10	5.4	3.7	1.5	0.2
81	5.5	2.4	3.7	1.0
133	6.3	2.8	5.1	1.5
137	6.4	3.1	5.5	1.8
75	6.6	3.0	4.4	1.4
109	7.2	3.6	6.1	2.5

```
In [13]: from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
print('Confusion Matrix: ')
print(cm)
ac1 = accuracy_score(y_test, y_pred)*100
print('Accuracy Score:')
print(ac1)
```

Confusion Matrix:

```
[[19  0  0]
 [ 0 14  1]
 [ 0  1 15]]
```

Accuracy Score:

96.0

In the multi-class classification problem, we won't get TP, TN, FP, FN values directly as in the binary classification problem. We need to calculate for each class REFER:

<https://www.analyticsvidhya.com/blog/2021/06/confusion-matrix-for-multi-class-classification/>

```
In [21]: #For Setosa Class
tp=cm[0][0]
fn=(cm[0][1])+(cm[0][2])
tn=(cm[1][1])+(cm[1][2])+(cm[2][1])+(cm[2][2])
fp=(cm[1][0])+(cm[2][0])
print('true positive: ',tp)
print('false positive: ',fp)
print('true negative: ',tn)
print('false negative: ',fn)
error_rate=(fp+fn)/(tp+tn+fp+fn)
print('error rate:', error_rate )
```

```
true positive: 19
false positive: 0
true negative: 31
false negative: 0
error rate: 0.0
```

```
In [22]: #For Versicolor Class
tp=cm[1][1]
fn=(cm[1][0])+(cm[1][2])
tn=(cm[0][0])+(cm[0][2])+(cm[2][0])+(cm[2][2])
fp=(cm[0][1])+(cm[2][1])
print('true positive: ',tp)
print('false positive: ',fp)
print('true negative: ',tn)
print('false negative: ',fn)
error_rate=(fp+fn)/(tp+tn+fp+fn)
print('error rate:', error_rate )
```

```
true positive: 14
false positive: 1
true negative: 34
false negative: 1
error rate: 0.04
```

```
In [23]: #For Virginica Class
tp=cm[1][2]
fn=(cm[2][0])+(cm[2][1])
tn=(cm[0][0])+(cm[0][1])+(cm[1][0])+(cm[1][1])
fp=(cm[0][2])+(cm[1][2])
print('true positive: ',tp)
print('false positive: ',fp)
print('true negative: ',tn)
print('false negative: ',fn)
error_rate=(fp+fn)/(tp+tn+fp+fn)
print('error rate:', error_rate )
```

```
true positive: 1
false positive: 1
true negative: 33
false negative: 1
error rate: 0.05555555555555555
```

```
In [20]: from sklearn.metrics import classification_report
print('
          classification report:')
print('')
print(classification_report(y_test,y_pred))
```

```

              classification report:

              precision    recall  f1-score   support

 Iris-setosa          1.00      1.00      1.00         19
 Iris-versicolor      0.93      0.93      0.93         15
 Iris-virginica       0.94      0.94      0.94         16

 accuracy              0.96         50
 macro avg             0.96         50
 weighted avg          0.96         50
```

```
In [7]: import seaborn as sns
```

```
In [11]: g=sns.relplot(x='SepalLengthCm',y='SepalWidthCm',data=df,hue='Species',style='Species')
g.fig.set_size_inches(10,5)
plt.show()
```

C:\Users\Shree\anaconda3\Lib\site-packages\seaborn\axisgrid.py:118: UserWarning: The figure layout has changed to tight
self._figure.tight_layout(*args, **kwargs)

